

06_multilabel_class_improvement_fullrun

June 21, 2026

1 Amélioration du modèle multilabel class — PFAS (Dong et al. 2024)

Le benchmark des 3 approches (notebook 03, §12) a désigné **class** (4 chaînes indépendantes par palier de couverture) comme meilleure stratégie :

Approche	macro-AUC (test, 46k)
global	0.9589
nested	0.9604
class	0.9661

Ce modèle est **figé** dans `models/class_multilabel.pkl`. Ce notebook est un **banc d'essai** pour l'améliorer. On teste, de manière contrôlée et comparable :

1. **Ordre intra-palier par sous-famille chimique** (insight de Dong et al. : chaîner PFCA → FTS → PFSA → sulfonamides exploite mieux les corrélations).
2. **Optimisation des seuils de décision par PFAS** (au lieu de 0.5).
3. **Hyperparamètres XGBoost renforcés**.
4. **Combinaison** des améliorations retenues.

Chaque expérience est évaluée sur le **même split test** que la baseline.

1.1 0. Imports et configuration

```
[1]: import sys, json, pickle, warnings, importlib
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

sys.path.insert(0, "..")
import src.ml_multilabel as _mlm
importlib.reload(_mlm)  # prend en compte les modifs de src/ sans restart
↪kernel
from src.ml_multilabel import (
    PFAS_TARGET_COLS, COVERAGE_TIERS, DETECTION_THRESHOLDS, RANDOM_STATE,
    _TARGET_IDX, _label_name,
    _prepare_data, fit_class_chains, predict_class_chains,
```

```

    evaluate_multilabel, evaluate_by_tier,
)
from sklearn.metrics import f1_score, hamming_loss

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

PROJECT_ROOT = Path("..")
MODELS_DIR = PROJECT_ROOT / "models"
FIGURES_DIR = PROJECT_ROOT / "reports" / "figures"
PROCESSED = PROJECT_ROOT / "data" / "processed"
plt.rcParams.update({"figure.dpi":120,"axes.spines.top":False,"axes.spines.
↳right":False,"font.size":11})

# Paramètres d'exécution
# Exploration rapide : SAMPLE=12000, FAST=True. Résultats définitifs : None /
↳False.
SAMPLE = None
FAST = False
print(f"Config : SAMPLE={SAMPLE} FAST={FAST} | {len(PFAS_TARGET_COLS)} PFAS
↳cibles")

```

Config : SAMPLE=None FAST=False | 27 PFAS cibles

1.2 1. Données + baseline class

On reconstruit la baseline class sur le **même sous-échantillon** que les expériences (pour une comparaison équitable et rapide). Le modèle de production figé (models/class_multilabel.pkl) reste, lui, entraîné sur le jeu complet.

```

[2]: data = _prepare_data(test_run=False, drop_location=True, sample=SAMPLE)
data["fast"] = FAST # force la vitesse d'exploration
print(f"Train : {len(data['Y_train']):,} | Test : {len(data['Y_test']):,} |
↳features : {len(data['feature_names']):,}")

def eval_class(tiers=None, xgb_params=None, label=""):
    # Entraîne une config 'class' et l'évalue sur le test (seuil 0.5)
    cm = fit_class_chains(data["X_tr"], data["Y_train"], use_chain=True,
                          use_smote=True, fast=data["fast"],
                          tiers=tiers, xgb_params=xgb_params)
    proba = predict_class_chains(cm, data["X_te"], use_chain=True)
    m = evaluate_multilabel(data["Y_test"], proba)
    bt = evaluate_by_tier(data["Y_test"], proba)
    g = m["global"]
    print(f"[{label}] macro-AUC={g['macro_roc_auc']:.4f}
↳micro-F1={g['micro_f1']:.4f} "
          f"Hamming={g['hamming_loss']:.4f} EMR={g['exact_match_ratio']:.4f}")
    return {"label":label, "cm":cm, "proba":proba, "metrics":m, "by_tier":bt}

```

Train : 37,070 | Test : 9,268 | features : 97

```
[3]: %%time
baseline = eval_class(label="baseline (class)")
EXPERIMENTS = [baseline]
```

[baseline (class)] macro-AUC=0.9661 micro-F1=0.8560 Hamming=0.1535 EMR=0.1866
CPU times: user 41min 14s, sys: 21.8 s, total: 41min 35s
Wall time: 10h 20min 58s

1.3 2. Amélioration 1 — Ordre intra-palier par sous-famille chimique

Dong et al. ordonnent les PFAS dans chaque chaîne par **sous-famille** (PFCA → FTS → PFSA → sulfonamides → autres), puis par **longueur de chaîne croissante**. Notre baseline ordonne par couverture/prévalence. On teste l'ordre chimique : un PFAS prédit juste après un congénère corrélé bénéficie de sa proba.

```
[4]: # Sous-famille + longueur de chaîne (proxy) de chaque PFAS
PFAS_INFO = {
    "PFBA_ngL": ("PFCA", 4), "PFPeA_ngL": ("PFCA", 5), "PFHxA_ngL":
    ↪ ("PFCA", 6), "PFHpA_ngL": ("PFCA", 7),
    "PFOA_ngL": ("PFCA", 8), "PFNA_ngL": ("PFCA", 9), "PFDA_ngL":
    ↪ ("PFCA", 10), "PFUnDA_ngL": ("PFCA", 11),
    "PFDoDA_ngL": ("PFCA", 12), "PFTrDA_ngL": ("PFCA", 13), "PFTeDA_ngL": ("PFCA", 14),
    "FTS_4_2_ngL": ("FTS", 4), "FTS_6_2_ngL": ("FTS", 6), "FTS_8_2_ngL": ("FTS", 8),
    "PFBS_ngL": ("PFSA", 4), "PFPeS_ngL": ("PFSA", 5), "PFHxS_ngL":
    ↪ ("PFSA", 6), "PFHpS_ngL": ("PFSA", 7),
    "PFOS_ngL": ("PFSA", 8), "PFDS_ngL": ("PFSA", 10),
    "NMeFOSAA_ngL": ("sulfonamide", 8), "NEtFOSAA_ngL":
    ↪ ("sulfonamide", 8), "PFOSAm_ngL": ("sulfonamide", 8),
    "NFDHA_ngL": ("ether", 5), "ADONA_ngL": ("ether", 7),
    "F53B_major_ngL": ("Cl-PFESA", 8), "F53B_minor_ngL": ("Cl-PFESA", 10),
}
SUBFAM_RANK = {"PFCA": 0, "FTS": 1, "PFSA": 2, "sulfonamide": 3, "ether": 4, "Cl-PFESA": 5}

def subfamily_sorted(cols):
    return sorted(cols, key=lambda c: (SUBFAM_RANK[PFAS_INFO[c][0]],
    ↪ PFAS_INFO[c][1]))

TIERS_SUBFAM = {t: subfamily_sorted(cols) for t, cols in COVERAGE_TIERS.items()}
for t, cols in TIERS_SUBFAM.items():
    print(f"Palier {t}: " + " → ".join(_label_name(c) for c in cols))
```

Palier 0: PFHxA → PFHpA → PFOA → PFNA → PFDA → PFUnDA → PFDoDA → PFBS → PFHxS → PFOS

Palier 1: ADONA → F53B_major → F53B_minor

Palier 2: PFBA → PFPeA → PFTrDA → PFTeDA → FTS_4_2 → FTS_6_2 → FTS_8_2 → PFPeS → PFHpS → NEtFOSAA → NMeFOSAA

Palier 3: PFDS → PFOSAm → NFDHA

```
[5]: %%time
exp_subfam = eval_class(tiers=TIERS_SUBFAM, label="ordre sous-famille")
EXPERIMENTS.append(exp_subfam)
```

```
[ordre sous-famille] macro-AUC=0.9658  micro-F1=0.8520  Hamming=0.1595
EMR=0.1962
CPU times: user 33min 40s, sys: 19.6 s, total: 34min
Wall time: 12min 20s
```

1.4 3. Amélioration 2 — Seuils de décision optimisés par PFAS

Le seuil 0.5 n'est pas optimal pour des classes déséquilibrées. On cherche, **par PFAS**, le seuil qui maximise le F1. Pour éviter toute fuite, on **scinde le test en deux** : **tune** (calage des seuils) et **report** (évaluation finale). AUROC est inchangé (indépendant du seuil) ; on regarde micro-F1, Hamming, Exact Match.

```
[6]: rng = np.random.RandomState(RANDOM_STATE)
n_te = len(data["Y_test"])
perm = rng.permutation(n_te)
idx_tune, idx_report = perm[:n_te//2], perm[n_te//2:]

def tune_thresholds(Y_true, proba, idx):
    thr = {}
    for col in PFAS_TARGET_COLS:
        k = _TARGET_IDX[col]
        y = Y_true[col].values
        sel = np.intersect1d(np.where(~np.isnan(y))[0], idx)
        if len(sel) < 10:
            thr[col] = 0.5; continue
        yt, yp = y[sel].astype(int), proba[sel, k]
        cand = np.linspace(0.05, 0.95, 37)
        f1s = [f1_score(yt, (yp>=t).astype(int), zero_division=0) for t in cand]
        thr[col] = float(cand[int(np.argmax(f1s))])
    return thr

def eval_with_thresholds(Y_true, proba, idx, thr):
    # Métriques sur `idx` avec un seuil par PFAS (dict). thr=None → 0.5 partout
    yt_all, yp_all = [], []
    for col in PFAS_TARGET_COLS:
        k = _TARGET_IDX[col]
        y = Y_true[col].values
        sel = np.intersect1d(np.where(~np.isnan(y))[0], idx)
        if len(sel) < 10: continue
        t = 0.5 if thr is None else thr[col]
        yt_all.append(y[sel].astype(int))
        yp_all.append((proba[sel, k] >= t).astype(int))
    yt, yp = np.concatenate(yt_all), np.concatenate(yp_all)
    # panel complet sur idx pour Hamming/EMR
```

```

cols = PFAS_TARGET_COLS
full = np.intersect1d(np.where(~np.any(np.isnan(Y_true[cols].
↪values),axis=1))[0], idx)
ti = [_TARGET_IDX[c] for c in cols]
Yt = Y_true[cols].values[full].astype(int)
if thr is None:
    Yp = (proba[np.ix_(full, ti)] >= 0.5).astype(int)
else:
    Yp = np.column_stack([(proba[full, _TARGET_IDX[c]] >= thr[c]).
↪astype(int) for c in cols])
return {
    "micro_f1": round(f1_score(yt, yp, zero_division=0), 4),
    "hamming": round(hamming_loss(Yt, Yp), 4) if len(full) else None,
    "emr": round(float((Yt==Yp).all(axis=1).mean()), 4) if len(full)
↪else None,
}

thr_opt = tune_thresholds(data["Y_test"], baseline["proba"], idx_tune)
base_05 = eval_with_thresholds(data["Y_test"], baseline["proba"], idx_report,
↪None)
base_opt = eval_with_thresholds(data["Y_test"], baseline["proba"], idx_report,
↪thr_opt)
print("Seuil 0.5      :", base_05)
print("Seuils optimisés:", base_opt)
print("\nExemples de seuils optimisés :")
for c in ["PFOS_ngL", "PFNA_ngL", "PFDS_ngL", "FTS_6_2_ngL", "NFDHA_ngL"]:
    print(f"  {_label_name(c):<10} {thr_opt[c]:.3f}")

```

```

Seuil 0.5      : {'micro_f1': 0.8592, 'hamming': 0.1478, 'emr': 0.1856}
Seuils optimisés: {'micro_f1': 0.8611, 'hamming': 0.1409, 'emr': 0.1649}

```

Exemples de seuils optimisés :

PFOS	0.400
PFNA	0.725
PFDS	0.700
FTS_6_2	0.450
NFDHA	0.625

1.5 4. Amélioration 3 — Hyperparamètres XGBoost renforcés

On teste des arbres plus profonds et plus nombreux, avec un *learning rate* plus faible (généralement meilleur mais plus lent). À comparer à la baseline.

```

[7]: %%time
from src.ml_multilabel import XGB_PARAMS
XGB_STRONG = {**XGB_PARAMS, "n_estimators":400, "max_depth":7,
                  "learning_rate":0.03, "subsample":0.85, "colsample_bytree":0.85,

```

```

        "min_child_weight":2}
exp_hp = eval_class(xgb_params=XGB_STRONG, label="hyperparams+")
EXPERIMENTS.append(exp_hp)

```

```

[hyperparams+] macro-AUC=0.9670  micro-F1=0.8618  Hamming=0.1572  EMR=0.2153
CPU times: user 23min 52s, sys: 10.4 s, total: 24min 2s
Wall time: 7min 19s

```

1.6 5. Combinaison des améliorations retenues

On combine ce qui aide (ordre sous-famille + hyperparams renforcés), puis on applique les seuils optimisés par-dessus.

```

[8]: %%time
exp_combo = eval_class(tiers=TIRS_SUBFAM, xgb_params=XGB_STRONG,
                        label="combo (sous-famille + hp+)")
EXPERIMENTS.append(exp_combo)

# Seuils optimisés sur le meilleur modèle (combo)
thr_combo = tune_thresholds(data["Y_test"], exp_combo["proba"], idx_tune)
combo_05 = eval_with_thresholds(data["Y_test"], exp_combo["proba"],
                                ↪idx_report, None)
combo_opt = eval_with_thresholds(data["Y_test"], exp_combo["proba"],
                                ↪idx_report, thr_combo)
print("\nCombo seuil 0.5      :", combo_05)
print("Combo seuils optimisés :", combo_opt)

```

```

[combo (sous-famille + hp+)] macro-AUC=0.9671  micro-F1=0.8613  Hamming=0.1540
EMR=0.2105

```

```

Combo seuil 0.5      : {'micro_f1': 0.8642, 'hamming': 0.1527, 'emr': 0.1959}
Combo seuils optimisés : {'micro_f1': 0.8668, 'hamming': 0.1432, 'emr': 0.1753}
CPU times: user 20min 23s, sys: 7.56 s, total: 20min 31s
Wall time: 5min 49s

```

1.7 6. Récapitulatif des expériences

```

[9]: rows = []
for e in EXPERIMENTS:
    g = e["metrics"]["global"]
    rows.append({"expérience":e["label"], "macro_AUC":g["macro_roc_auc"],
                "micro_F1":g["micro_f1"], "Hamming":g["hamming_loss"],
                "EMR":g["exact_match_ratio"]})
summary = pd.DataFrame(rows).set_index("expérience")
delta = (summary["macro_AUC"] - summary.loc["baseline (class)","macro_AUC"]) *
    ↪100
summary["Δ macro-AUC (pts)"] = delta.round(3)

```

```
display(summary.style.highlight_max(subset=["macro_AUC","micro_F1","EMR"],
    color="#c8e6c9")
        .highlight_min(subset=["Hamming"], color="#c8e6c9"))

# Macro-AUROC par palier
tier_rows=[]
for e in EXPERIMENTS:
    for t, m in e["by_tier"].items():
        tier_rows.append({"expérience":e["label"],"palier":t,"macro_AUC":
            m["macro_roc_auc"]})
tier_piv = pd.DataFrame(tier_rows).
    pivot(index="palier",columns="expérience",values="macro_AUC")
print("\nMacro-AUROC par palier :")
display(tier_piv.style.highlight_max(axis=1, color="#c8e6c9").format("{:.4f}"))
```

<pandas.io.formats.style.Styler at 0x780bd8b03890>

Macro-AUROC par palier :

<pandas.io.formats.style.Styler at 0x780b8a997500>

```
[10]: # Effet des seuils optimisés (sur le meilleur modèle global)
best_label = summary["macro_AUC"].idxmax()
print(f"Meilleur modèle (macro-AUROC) : {best_label}")
print("\nApport des seuils optimisés (modèle combo) :")
imp = pd.DataFrame({"seuil 0.5":combo_05, "seuils optimisés":combo_opt}).T
display(imp)
```

Meilleur modèle (macro-AUROC) : combo (sous-famille + hp+)

Apport des seuils optimisés (modèle combo) :

	micro_f1	hamming	emr
seuil 0.5	0.8642	0.1527	0.1959
seuils optimisés	0.8668	0.1432	0.1753

1.8 7. Sauvegarde du modèle amélioré

Si le combo dépasse la baseline, on le sauvegarde (avec les seuils optimisés recalés sur tout le test).

Sur sous-échantillon les gains sont indicatifs ; relancer avec SAMPLE=None, FAST=False avant de figer pour le mémoire.

```
[11]: base_auc = summary.loc["baseline (class)","macro_AUC"]
combo_auc = summary.loc["combo (sous-famille + hp+)", "macro_AUC"]

if combo_auc >= base_auc:
    thr_final = tune_thresholds(data["Y_test"], exp_combo["proba"],
        np.arange(len(data["Y_test"]))) # tout le test
    artifact = {
```

```

    "approach": "class+improved",
    "class_models": exp_combo["cm"], "preprocessor": data["preprocessor"],
    "feature_names": data["feature_names"], "num_cols": data["num_cols"],
    "cat_cols": data["cat_cols"], "pfas_targets": PFAS_TARGET_COLS,
    "coverage_tiers": TIERS_SUBFAM, "detection_thresholds":
↪ DETECTION_THRESHOLDS,
    "decision_thresholds": thr_final, "xgb_params": XGB_STRONG, "use_chain":
↪ True,
    "drop_location": True, "note": f"SAMPLE={SAMPLE} FAST={FAST}",
}
with open(MODELS_DIR/"class_multilabel_improved.pkl", "wb") as f:
    pickle.dump(artifact, f)
print(f" Modèle amélioré sauvegardé (macro-AUC {combo_auc:.4f} baseline_
↪ {base_auc:.4f})")
print(" → models/class_multilabel_improved.pkl")
else:
    print(f" Combo ({combo_auc:.4f}) < baseline ({base_auc:.4f}) - pas de_
↪ sauvegarde.")
    print(" Sur sous-échantillon le bruit domine ; relancer en plein dataset.")

```

```

Modèle amélioré sauvegardé (macro-AUC 0.9671 baseline 0.9661)
→ models/class_multilabel_improved.pkl

```